# CS 188 Fall 2025 — Intro to Artificial Intelligence Final Exam

**Solutions last updated: Monday, April 27, 2026**

PRINT Your Name: ______________________________

PRINT Your Student ID: ______________________________

PRINT Student name to your left: ______________________________

PRINT Student name to your right: ______________________________

You have 170 minutes. There are 8 questions of varying credit. (100 points total)

| Question: | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | Total |
|---|---|---|---|---|---|---|---|---|---|
| Points: | 5 | 6 | 15 | 17 | 17 | 17 | 17 | 6 | 100 |

We reserve the right to deduct points for failing to follow the marking directions below:

For questions with **circular bubbles**, you may select only one choice.

- Ⓐ Unselected option (Completely unfilled)
- Ⓑ Don't do this (it will be graded as incorrect!)
- ● Only one selected option (completely filled)

For questions with **square checkboxes**, you may select one or more choices.

- ■ You can select
- ■ multiple squares
- ☑ Don't do this (it will be graded as incorrect!)

Anything you write outside the answer boxes or you ~~cross out~~ will not be graded. If you write multiple answers, your answer is ambiguous, or the bubble/checkbox is not entirely filled in, we will grade the worst interpretation.

Read the honor code below and sign your name.

> By signing below, I affirm that all work on this exam is my own work. I have not referenced any disallowed materials, nor collaborated with anyone else on this exam. I understand that if I cheat on the exam, I may face the penalty of an "F" grade and a referral to the Center for Student Conduct.

SIGN your name: ______________________________

# Clarifications

- Q5.4: To randomly choose between A1, A2, and B1, the alphabetical order is A1, A2, B1.

# Q1 *Potpourri* (5 points)

Q1.1 and Q1.2 are from the guest lectures.

**Solution:** Note for future semesters: These guest lectures are specific to Fall 2025, so these subparts are probably out of scope for future semesters.

Q1.1 (1 point) In Prof. Alaa's guest lecture, he talks about two metrics that can be used to compare real and synthetic distributions. What are the two metrics that he brings up?

- [ ] A Precision
- [x] B Fidelity
- [x] C Diversity
- [ ] D Recall

**Solution:**

See https://inst.eecs.berkeley.edu/~cs188/fa25/assets/lectures/cs188-fa25-lec25.pdf#page=20.

Q1.2 (1 point) In Prof. Pierson's guest lecture, she discusses two specific examples of using AI to create social equity. What are the two examples discussed?

- [x] A Knee osteoarthritis
- [ ] B Deforestation
- [x] C Inequality in policing
- [ ] D Inequality in STEM education

**Solution:**

See https://inst.eecs.berkeley.edu/~cs188/fa25/assets/lectures/cs188-fa25-lec26.pdf.

For Q1.3 to Q1.5, consider an arbitrary Bayes Net with no edges. $X$, $Y$, and $Z$ represent arbitrary variables in the Bayes Net.

Q1.3 (1 point) In a Bayes Net with no edges, what is the most efficient way to calculate a conditional probability like $P(X \mid Y)$, with **one** query variable and **one** evidence variable?

- (●) A Return one of the CPTs (Conditional Probability Tables) with no changes.
- ( ) B Perform one or more join operations only.
- ( ) C Perform one or more eliminate operations only.
- ( ) D Perform one or more join operations, and one or more eliminate operations.

**Solution:**

If our Bayes Net has no edges, then all variables are independent of each other. Each node has a CPT involving only that node's variable, e.g. node $X$ has the CPT $P(X)$.

So, $P(X \mid Y) = P(X)$, so all you need to return is $P(X)$.

Q1.4 (1 point) In a Bayes Net with no edges, what is the most efficient way to calculate a conditional probability like $P(X \mid Y, Z)$, with **one** query variable and **multiple** evidence variables?

- (A) Return one of the CPTs with no changes.
- (B) Perform one or more join operations only.
- (C) Perform one or more eliminate operations only.
- (D) Perform one or more join operations, and one or more eliminate operations.

**Solution:** If our Bayes Net has no edges, then all variables are independent of each other. Each node has a CPT involving only that node's variable, e.g. node $X$ has the CPT $P(X)$.

So, $P(X \mid Y, Z) = P(X)$, so all you need to return is $P(X)$.

Q1.5 (1 point) In a Bayes Net with no edges, what is the most efficient way to calculate a conditional probability like $P(X, Y \mid Z)$, with **multiple** query variables and **one** evidence variable?

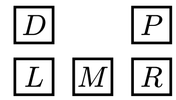
- (A) Return one of the CPTs with no changes.
- (B) Perform one or more join operations only.
- (C) Perform one or more eliminate operations only.
- (D) Perform one or more join operations, and one or more eliminate operations.

**Solution:** Since all variables are independent of each other, $P(X, Y \mid Z) = P(X, Y)$. To get $P(X, Y)$, we need to join $P(X)$ and $P(Y)$.

## Q2 *CSP: Car Seat Planning* **(6 points)**

CS188 staff is going on a road trip! There are five possible seats for staff members to sit in the car. The staff members are the variables, and the seats are the values. No two staff members can sit in the same seat.

The staff members have special requests on which seat they want to sit in:



1. Only Isabella and Kanav can sit in the driver's seat $D$.
2. Andrew does not want to sit in the middle seat $M$.
3. Aly gets carsick and needs to sit in a seat next to a window ($D$, $P$, $L$, or $R$).
4. Saathvik wants to sit one seat to the left or right of Kanav, i.e., they can sit in ($D$, $P$), ($L$, $M$), or ($M$, $R$).

Q2.1 (1 point) After enforcing only unary constraints, which of these arcs is already consistent, without removing additional values from any domains?

(A) (Kanav $\to$ Saathvik) only  (B) (Saathvik $\to$ Kanav) only  (C) Both  (D) Neither

**Solution:** (Saathvik $\to$ Kanav) is consistent because Saathvik's domain after enforcing unary constraints is $\{P, L, M, R\}$ and for each of those possible values, Kanav still has at least one neighboring seat available in his domain. (Kanav $\to$ Saathvik) is not consistent because if Kanav sits in $P$, Saathvik must sit in $D$, which violates the first request.

Q2.2 (1 point) After enforcing only unary constraints, we use LCV (Least Constraining Value) with forward checking to assign Kanav first. Select all value(s) that LCV could assign to Kanav.

[A] $D$  [B] $P$  [C] $L$  [D] $M$  [E] $R$

**Solution:** If Kanav is assigned to $M$, Saathvik can still be assigned to 2 values $L$ or $R$. If Kanav is assigned to $P$, Saathvik cannot be assigned to any value, and if Kanav is assigned to $D$, $L$, or $R$, Saathvik can only be assigned to 1 value.

For Q2.3 to Q2.5, we use cutset conditioning to solve this CSP, where the cutset is Saathvik and Kanav. In cutset conditioning, after we assign values to one or more variables, the result is a *residual CSP*, which is a smaller CSP with only the unassigned variables left.

Assume we create residual CSPs by assigning values to Saathvik and Kanav, and **not** checking any constraints (except the "no two staff members in the same seat" constraint) when assigning values to Saathvik and Kanav.

Q2.3 (1 point) How many total residual CSPs do we need to create in order to find all solutions to the original CSP?

(A) 1  (B) 2  (C) 3  (D) 6  (E) 10  (F) 20

**Solution:** We have 20 possible residual CSPs after assigning Saathvik and Kanav because Kanav can sit in any of the 5 seats and Saathvik can choose from the 4 remaining options.

Q2.4 (2 points) Select all true statements about each residual CSP.

- [A] Each residual CSP has a tree-structured constraint graph.
- [B] Each residual CSP has a constraint graph with no edges.
- [C] Each residual CSP only has unary constraints (except "no two staff members in the same seat").
- [D] Each residual CSP can be solved with no backtracking.
- [E] Each residual CSP takes worst-case exponential time to solve (i.e., trying every assignment).
- (F) None of the above

**Solution:** The remaining three constraints after assigning Kanav and Saathvik to a seat are all unary. The residual CSP has a constraint graph with no edges because we only have unary constraints left. The residual CSP can be solved without backtracking because we can enforce unary constraints and then solve accordingly.

Q2.5 (1 point) Does a solution for a residual CSP correspond to a solution for the original CSP?

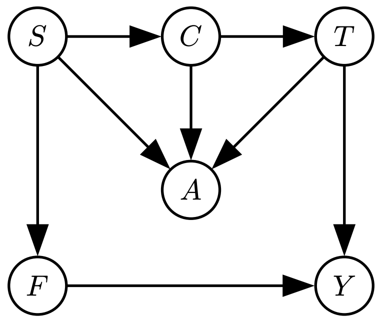
(A) Always (B) Sometimes (C) Never

**Solution:** Our residual CSP may place Saathvik and Kanav in a configuration that is invalid for our original CSP (i.e., not in adjacent seats).

## Q3 *Bayes Nets: Donut Production Line* **(15 points)**

Pac-Man is making donuts and models their attributes using a Bayes Net. Each donut has a shape $S$, icing color $C$, topping $T$, filling $F$, yumminess $Y$, and aesthetics $A$.

All variables in the Bayes Net are binary (i.e., each variable can take on two possible values).




For Q3.1 to Q3.4, use $d$-separation to determine if the independence assumption is true or false.

Q3.1 (1 point) $F \perp\!\!\!\perp T$ Ⓐ True Ⓑ False

Q3.2 (1 point) $A \perp\!\!\!\perp Y \mid T$ Ⓐ True Ⓑ False

Q3.3 (1 point) $S \perp\!\!\!\perp T \mid C$ Ⓐ True Ⓑ False

Q3.4 (1 point) $F \perp\!\!\!\perp T \mid S$ Ⓐ True Ⓑ False

**Solution:** We have an active path from $F \leftarrow S \rightarrow C \rightarrow T$, which creates dependence, as this is an example of a common effect.

$A \leftarrow T \rightarrow Y$ blocks the direct path, but $A \leftarrow S \rightarrow F \rightarrow Y$ is a path that remains open. This path leads to dependence, as this is a common effect.

$S \rightarrow C \rightarrow T$ forms a causal chain. Because $C$ is given, we have an inactive triple, so we have independence.

$F \leftarrow S \rightarrow C \rightarrow T$ has $S$ as a common cause, and because $S$ is given, we know that they are independent of each other.

Q3.5 to Q3.10 are independent of each other.

Q3.5 (1 point) We use inference by enumeration to compute $P(T \mid +f)$. What is the size of the largest factor generated?

Ⓐ 1 Ⓑ 2 Ⓒ 4 Ⓓ 8 Ⓔ 16 Ⓕ 32 Ⓖ 64

**Solution:** Join all variables – 5 variables total. (+f is evidence)

Q3.6 (2 points) We use variable elimination to compute $P(T \mid +f)$. We join and eliminate on $S$ first. Which CPTs (Conditional Probability Tables) do we need to use when joining and eliminating on $S$? Select all that apply.

- [x] A $P(S)$
- [x] B $P(C \mid S)$
- [x] C $P(A \mid S, C, T)$
- [ ] D $P(T \mid C)$
- [x] E $P(+f \mid S)$
- [ ] F $P(Y \mid +f, T)$

**Solution:** We use $P(S), P(C \mid S), P(A \mid S, C, T), P(+f \mid S)$ since these are all the CPTs that use $S$.

Q3.7 (2 points) We use sampling to approximate $P(T \mid +f, -a)$. Which sampling techniques could have generated the following sequence of samples? Select all that apply.

Sample 1: $(-s, +c, +t, +f, +y, -a)$.

Sample 2: $(-s, -c, -t, -f, +y, -a)$.

Sample 3: $(+s, +c, -t, +f, +y, -a)$.

- [x] A Prior Sampling
- [ ] B Rejection Sampling
- [ ] C Likelihood Weighting
- [ ] D Gibbs Sampling
- ( ) E None of the above

**Solution:** Prior sampling is the only sampling method that can generate samples inconsistent with evidence. Sample 2 has $-f$, so it is inconsistent with evidence.

Q3.8 (2 points) We use likelihood weighting to approximate $P(T \mid +f, -a)$. We generate the following samples:

Sample 1: $(+s, +c, -t, +f, -y, -a)$.

Sample 2: $(-s, +c, -t, +f, +y, -a)$.

Sample 3: $(+s, -c, +t, +f, -y, -a)$.

Which CPTs do we need in order to calculate the weights of the samples? Select all that apply.

- [ ] A $P(S)$
- [ ] B $P(C \mid S)$
- [x] C $P(-a \mid S, C, T)$
- [ ] D $P(T \mid C)$
- [x] E $P(+f \mid S)$
- [ ] F $P(Y \mid +f, T)$

**Solution:** When we calculate the weights, we only need P(evidence | parents(evidence)).

Q3.9 (2 points) We use Gibbs sampling to approximate $P(T \mid +a)$. The last generated sample is $(+s, -c, -t, -f, -y, +a)$.

We choose to resample $T$. Which CPTs do we need to join in order to resample $T$? Select all that apply.

- [ ] A $P(S)$
- [x] C $P(A \mid +s, -c, -t)$
- [ ] E $P(F \mid +s)$
- [ ] B $P(C \mid +s)$
- [x] D $P(T \mid -c)$
- [x] F $P(Y \mid -f, -t)$

**Solution:** You just have to join the CPTs that include $T$ or a value for $T$.

Q3.10 (2 points) We use Gibbs sampling to approximate $P(T \mid +a)$. The last generated sample is $(+s, -c, -t, -f, -y, +a)$.

Which of these distributions can be used **by itself** to resample $C$? There may be multiple answers; select all equivalent answers that work.

- [ ] A $P(C)$
- [x] C $P(C \mid +a, +s, -t)$
- [ ] E $P(Y \mid -c)$
- [ ] B $P(C \mid +s)$
- [x] D $P(C \mid +a, +s, -t, -f, -y)$
- [ ] F $P(Y \mid -f, -t)$

**Solution:** Choices $C$ and $D$ are both true as they give the conditional distribution of $C$ given its Markov blanket. Choice $C$ is correct as it conditions on the full Markov blanket. Choice $D$ is correct as it conditions on the superset of the Markov blanket, meaning it includes extra variables ($y$, $c$, and $f$).

## Q4 *MDP: Schrodinger's Gridworld!* **(17 points)**

Consider the Gridworld MDP to the right, with starting state $Y$, discount factor $\gamma = 1$, and living reward 0.

| | | |
|---|---|---|
| $+10$ | | $-10$ |
| $X$ | $Y$ | $Z$ |

Remember that in Gridworld, when we are in the $+10$ or $-10$ square, the only action available is "Exit," which succeeds with 100% probability, and we must take the "Exit" action to earn the reward.

In state $Y$, the only available actions are "Left" and "Right". With 70% probability, the action succeeds, and with 30% probability, the opposite action is taken (e.g., if we go Left from $Y$, then we land in $Z$ 30% of the time).

Q4.1 (1 point) We are running value iteration. On what iteration $k$ does $V_k(Y)$ first become nonzero?

(A) 0 (B) 1 (C) 2 (D) 3

**Solution:** At the 0th iteration, Y's value is 0. After the first iteration, we can move to states $X$ or $Z$, but we only get their value after taking the Exit action. At the second iteration, we take the Exit action on state $X$, making $V_2(Y)$ nonzero.

Q4.2 (1 point) What is the value of $V^*(Y)$? Write a number in the box.

4

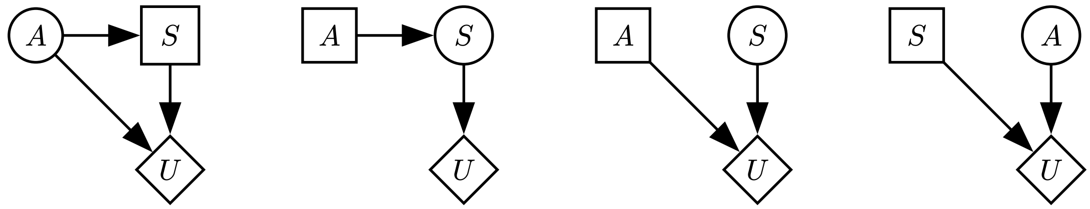
**Solution:**

With probability 70%, we get the $+10$, and with probability 30%, we get the $-10$.

$V^*(X) = +10(0.70) - 10(0.30) = 4$

This Gridworld can also be modeled as a decision network, where:
- $A$ is the action taken.
- $S$ is whether the action succeeds (True for success, and False for failure).
- $U$ is the reward.

Q4.3 (1 point) Which of the following decision networks can model this scenario?




Ⓐ Network (i) Ⓑ Network (ii) Ⓒ Network (iii) Ⓓ Network (iv)

**Solution:**

Choice B is incorrect because the action taken and the action success both influence the utility.

Choice D is incorrect because $S$ is incorrectly represented by an action node and $A$ is incorrectly represented by a chance node.

Choice A and B are incorrect because action nodes should not directly point to chance nodes as actions should be made on deterministic outcomes.

Q4.4 (2 points) Fill in $U(A, S)$, the utilities in the decision network. Write one number in each box.

| $A$ | $S$ | $U(A, S)$ |
|---|---|---|
| Left | True | |
| Left | False | |

| $A$ | $S$ | $U(A, S)$ |
|---|---|---|
| Right | True | |
| Right | False | |

**Solution:**

| $A$ | $S$ | $U(A, S)$ |
|---|---|---|
| Left | True | $+10$ |
| Left | False | $-10$ |
| Right | True | $-10$ |
| Right | False | $+10$ |

If you intend to go Left and it succeeds, then you reach state $X$ and get $+10$. If you intend to go Left and it fails, then you reach state $Z$ and get $-10$. If you intend to go Right and it succeeds, then you reach state $Z$ and get $-10$. Finally, if you intend to go Right and it fails, then you reach state $X$ and get $+10$.

Q4.5 (2 points) Compute MEU($\varnothing$) for the decision network. Write a number in the box.

4

**Solution:**

If we move left, with probability 70%, we get the +10, and with probability 30%, we get the −10. The expected utility is 0.7(+10) + 0.3(-10) = 4

If we move right, with probability 70%, we get the −10, and with probability 30%, we get the +10. The expected utility is 0.7(-10) + 0.3(+10) = −4

Therefore, the MEU($\varnothing$) is max(4, −4) = 4.

Reminders: Starting state is $Y$, discount factor is $\gamma = 1$, and living reward is 0.

| | | |
|---|---|---|
| +10 | | −10 |
| $X$ | $Y$ | $Z$ |

| | | |
|---|---|---|
| −10 | | +10 |
| $X$ | $Y$ | $Z$ |

For the rest of the question, consider the *Schrodinger's Gridworld* problem: We are in one of the two Gridworlds shown, with a 50% probability of being in either Gridworld. Initially, we do not know which Gridworld we are in.

In state $Y$, the three available actions are:

- Left and Right, which succeed with 70% probability, as before.
- Observe, which reveals which Gridworld we are in and keeps us in state $Y$. This action succeeds 100% of the time.

Q4.6 (2 points) Intuitively, we know that taking the Observe action multiple times will not increase our expected utility. How could we modify the problem such that an optimal agent would not take the Observe action multiple times? Consider each modification separately.

- [x] A. Add a negative reward for taking the Observe action.
- [ ] B. Add a positive living reward.
- [ ] C. Change the probability of success for the Observe action to 70%. If the Observe action fails, we gain no additional information and stay in state $Y$.
- [x] D. Change the discount factor to be less than 1.
- ( ) E. None of the above

**Solution:** A negative reward will help because by taking the Observe action multiple times, the agent will unnecessarily incur additional negative reward.

A positive living reward will make the agent repeatedly take the Observe action so that they never have to exit and keep accumulating the living reward.

If we change the probability of success for the Observe action to 70%, because the discount factor is 1, the agent will just keep trying to Observe from state Y even if it fails.

Having a discount factor less than 1 will make the agent prefer short-term rewards over long-term ones. This makes our agent prefer to move to states $X$ or $Z$ instead of taking an unnecessary Observe action.

Q4.7 (2 points) In this subpart, ignore the modifications in the previous subpart.

An agent (who has not yet taken any actions) is deciding between two strategies:
- Strategy 1: Observe, then go Left or Right towards the state with the $+10$ reward.
- Strategy 2: Do not Observe, and always go Left.

What should the reward for Observe be, such that the two strategies have the same expected return? Write a number in the box.

$-4$

**Solution:** Let $L$ represent the state with the $+10$ reward. The value of $L$ is initially unknown.

$\text{VPI}(L) = 0.5 * \text{VPI}(L = X) + 0.5 * \text{VPI}(L = Z) = \text{VPI}(L = X)$ (by symmetry)

$\text{VPI}(L = X) = \text{MEU}(L = X) - \text{MEU}(\varnothing)$

$\text{MEU}(\varnothing) = 0$ by symmetry.

$\text{MEU}(L = X) \to$ we take the action Left. Reward is 4.

$\text{VPI}(L) = 4$.

So, we would pay 4 (negative reward of 4) to take the Observe action.

Consider modeling the *Schrodinger's Gridworld* problem as an MDP with the following state space:
- $Y_0, Y_1, Y_2$ represents the agent in state $Y$ without observing, after observing $+10$ in $X$, and after observing $+10$ in $Z$, respectively.
- $X_0, X_1, X_2$ and $Z_0, Z_1, Z_2$ are defined similarly.

In this MDP, all rewards are 0, except taking an Exit action.

Q4.8 (2 points) What is $Q^*(Y_0, \text{Observe})$?

Ⓐ 0 **Ⓑ 4** Ⓒ 5 Ⓓ 7 Ⓔ 10

**Solution:** After we observe, the optimal action will be to go Left or Right depending on if X or Z will give us the +10 utility. With 70% probability, moving in the direction to the +10 succeeds and lands us in the +10 square. With 30% probability, moving in the direction to the +10 fails and lands us in the −10 square. So, we have (0.7)(10) + (0.3)(-10) = 4.

Q4.9 (2 points) What is $Q^*(Y_0, \text{Left})$?

**Ⓐ 0** Ⓑ 4 Ⓒ 5 Ⓓ 7 Ⓔ 10

**Solution:** From $Y_0$, there is a 50% chance that we will get a +10 reward and a 50% chance we will get a −10 reward, due to symmetry.

Q4.10 (2 points) What is $Q^*(Y_1, \text{Left})$?

Ⓐ 0 **Ⓑ 4** Ⓒ 5 Ⓓ 7 Ⓔ 10

**Solution:** $Y_1$ means that the $+10$ reward is in $X$.

With $70\%$ probability, moving Left succeeds and lands us in the $+10$ square.

With $30\%$ probability, moving Left fails and lands us in the $-10$ square.

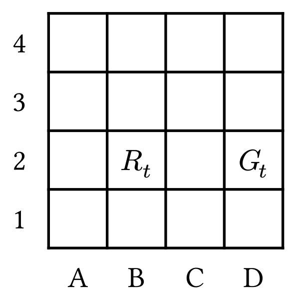
Taking a weighted average gives an expected reward of $0.7(10) + 0.3(-10) = 4$.

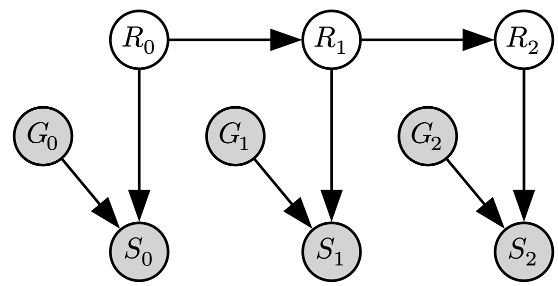
## Q5 *HMMs: Heist Money Moves* **(17 points)**

In the Louvre museum, a robber is being chased by a guard! The robber's position ($R_t$) and the guard's position ($G_t$) are positions on a $4 \times 4$ grid, shown below.

At each time step $t$:
- The robber moves up, down, left, or right with equal probability. If the robber moves in a direction that hits a wall, then the robber stays in the same square.
- The guard instantly moves to a random square (not necessarily an adjacent square).
- The guard reports a sighting $S_t$ of the robber. The value of $S_t$ is `near` if the guard is Manhattan distance 0 or 1 from the robber, and `far` otherwise. $S_t$ is correctly reported with probability 0.9.

$R_t$ is the hidden variable, and $G_t$ and $S_t$ are the evidence. We can model this problem as the HMM below:




Example:
- At $t = 10$, suppose $R_{10} = \text{B2}$ and $G_{10} = \text{D2}$.
- The robber moves up to $R_{11} = \text{B3}$, and the guard moves to $G_{11} = \text{B4}$.
- $S_{11}$ is `near` with probability 0.9 and `far` otherwise.

Q5.1 (2 points) In the HMM, which expression represents the belief distribution at time $t$, after incorporating the evidence at time $t$?

(A) $P(R_{t+1}, G_t, S_t)$  (B) $P(R_t \mid G_t, S_t)$  (C) $P(R_{t+1}, G_{0:t}, S_{0:t})$  (D) [selected] $P(R_t \mid G_{0:t}, S_{0:t})$

**Solution:** Belief distribution is $P(R_t \mid G_{0:t}, S_{0:t})$

Q5.2 (2 points) Which of these distributions can be used **by itself** to perform a time elapse update in the forward algorithm? There may be multiple answers; select all answers that work.

- [x] A $P(R_{t+1} \mid R_t)$
- [x] B $P(R_{t+1} \mid R_t, S_t, G_t)$
- [ ] C $P(R_{t+1} \mid S_{t+1}, G_{t+1})$
- [ ] D $P(R_{t+1} \mid S_{t+1})$
- (E) None of the above

**Solution:** Transition model is $P(R_{t+1} \mid R_t)$. $P(R_{t+1} \mid R_t, S_t, G_t)$ works too because $R_{t+1}$ is conditionally independent of $S_t$ and $G_t$ given $R_t$.

Q5.3 (2 points) Which of these distributions can be used **by itself** to perform an observation update in the forward algorithm? There may be multiple answers; select all answers that work.

- [x] A $P(S_t, G_t \mid R_t)$
- [ ] B $P(S_t \mid G_t)$
- [x] C $P(S_t \mid G_t, R_t)$
- [ ] D $P(S_t, G_t \mid S_{t-1}, G_{t-1})$
- ( ) E None of the above

**Solution:** Observation model is

$$\begin{aligned} P(S_t, G_t \mid R_t) &= P(S_t \mid G_t, R_t) \cdot P(G_t \mid R_t) \\ &= P(S_t \mid G_t, R_t) \cdot P(G_t) \\ &\propto P(S_t \mid G_t, R_t) \end{aligned}$$

Now, suppose we use particle filtering with 3 particles to estimate the robber's location over time.

Q5.4 (2 points) At $t = 1$, we have the three particles [A4, A4, D4]. After applying a time elapse update, what squares are the particles in?

Use the following randomly-generated numbers for sampling (you may not need all of them): 0.94, 0.61, 0.12, 0.58, 0.20, 0.81

When sampling, split the 0 to 1 range alphabetically, e.g., to randomly choose between A1 and A2, low numbers correspond to A1 and high numbers correspond to A2.

- ( ) A [A3, A3, C4]
- ( ) B [A3, A4, C4]
- ( ) C [A3, A4, D3]
- ( ) D [A4, B4, D3]
- ( ) E [A4, B4, D4]
- ( ) F [B4, A3, C4]
- ( ) G [B4, A3, D3]
- (●) H [B4, A4, C4]

**Solution:** A4 particle successors are $A3, A4, B4$ with probabilities $0.25, 0.5, 0.25$. So particle 1 goes to B4 (0.94>0.75), particle 2 goes to A4 (0.25 < 0.61 < 0.75). Door particle successors are $C4, D3, D4$ with probabilities $0.25, 0.25, 0.5$ so particle 3 goes to C4 (0.12 < 0.25).

Q5.5 (2 points) At $t = 2$, suppose that $G_2 = \text{C3}$ and that $S_2$ is `far`.

Consider a particle at square B3 at $t = 2$. What is the weight of this particle after applying the observation update?

- ( ) A 0
- (●) B 0.1
- ( ) C 0.2
- ( ) D 0.25
- ( ) E 0.45
- ( ) F 0.5
- ( ) G 0.75
- ( ) H 0.8
- ( ) I 0.9
- ( ) J 1

**Solution:** $P(S_2 = \text{far} \mid R_2 = B2, G_2 = C2) = 0.1$ since $B2$ and $C2$ are adjacent cells, and the guard will incorrectly report a sighting distance with probability 0.1.

Q5.6 (2 points) At $t = 3$, before resampling, 2 particles are in C1, each with weight $x$, and 1 particle is in B1 with weight $y$.

When we resample, what is the probability that the first particle we resample will be in B1? Write an expression in terms of $x$, $y$, and any mathematical operators.

$\frac{y}{2x+y}$

**Solution:** The total weights for state C1 is $2x$, the total weights for state B1 is $y$, and the total weights for any other state is 0. We can normalize this distribution to get that that the probability a particle will be resampled to be in state B1 is $\frac{y}{2x+y}$.

In Q5.7 to Q5.9, we run particle filtering for a long time, and at some point, we notice that all of our particles are in D1. Each subpart is independent.

Q5.7 (1 point) Immediately after a re-sampling step, all particles are in D1.
Is the robber guaranteed to be in D1 at this time step?

(A) Yes

(B) No

**Solution:** Particle filtering is a representation of our beliefs. Even though we believe that the robber is on D1, there is no guarantee that they are in this state because this is an approximation.

Q5.8 (2 points) Immediately after a time elapse update, all particles are in D1.
After we perform an observation update, all particles ____ get the same weight (before re-sampling).

(A) Always

(B) Sometimes

(C) Never

**Solution:** The weight depends on only the state and the observation, so all particles in the same state are assigned the same weight.

Q5.9 (2 points) Immediately after a re-sampling step, all particles are in D1.
After we perform another time elapse and observation update, all particles are ____ in the same square (not necessarily D1).

(A) Always

(B) Sometimes

(C) Never

**Solution:** After the time elapse, there is no guarantee that all of the particles will move into the same successor state. However, it is possible that all particles will randomly move into the same successor state.

## Q6 *Machine Learning: Classification* **(17 points)**

For Q6.1 and Q6.2, consider the training data in the table, with features $X_1$, $X_2$, and label $Y$. We build a Naive Bayes model using this training data.

| $X_1$ | $X_2$ | $Y$ |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 0 | 0 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |
| 1 | 1 | 1 |
| 1 | 1 | 1 |

Q6.1 (1 point) Which CPTs (Conditional Probability Tables) live in the Bayes Net for this model? Select all that apply.

- [x] A $P(Y)$
- [x] B $P(X_1 \mid Y)$
- [ ] C $P(Y \mid X_2)$
- [ ] D $P(Y \mid X_1, X_2)$
- ( ) E None

**Solution:** Under a Naive Bayes model, we assume that all features are dependent on our label and conditionally independent given the label. This means that we require the CPTs $P(Y)$, $P(X_1 \mid Y)$, and $P(X_2 \mid Y)$.

Q6.2 (2 points) Using the Naive Bayes model, what is the classification for a new data point with $X_1 = 1$ and $X_2 = 0$?

- ( ) A $\hat{Y} = 0$
- (●) B $\hat{Y} = 1$
- ( ) C None of these

**Solution:** We compute

$P(X_1 = 1, X_2 = 0, Y = 0) = P(X_1 = 1 \mid Y = 0) \cdot P(X_2 = 0 \mid Y = 0) \cdot P(Y = 0) = \frac{2}{3} \cdot \frac{2}{3} \cdot \frac{3}{8} = \frac{1}{6}$

$P(X_1 = 1, X_2 = 0, Y = 1) = P(X_1 = 1 \mid Y = 1) * P(X_2 = 0 \mid Y = 1) * P(Y = 1) = \frac{4}{5} \cdot \frac{2}{5} \cdot \frac{5}{8} = \frac{1}{5}$

Since $\frac{1}{6} < \frac{1}{5}$, our prediction is $Y = 1$.

Q6.3 (2 points) In general, what strategies can help a Naive Bayes model better generalize to unseen data? Select all that apply.

- **A** Use Laplace smoothing.
- B Use maximum likelihood estimation to estimate the CPTs.
- C Reduce the number of training data points.
- D Normalize the probabilities in the CPTs.
- E None of the above

**Solution:** Laplace smoothing allows us to mitigate overfitting for Naive Bayes' classifiers.

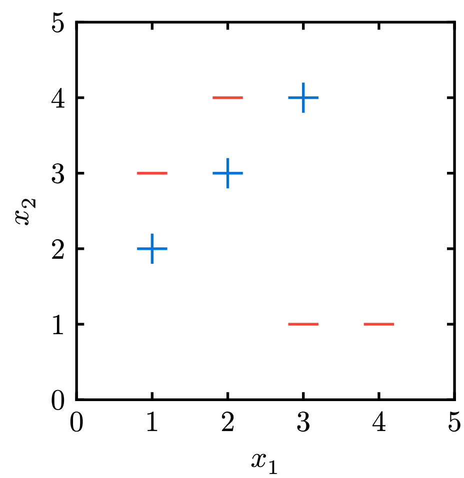
MLE would not help our model generalize to unseen data.

Reducing the training data size gives our classifer even less information to train on which may make it harder for our model to generalize on unseen data.

Normalizing the probabilities in our CPT does not change anything as they're already normalized.

For Q6.4 to Q6.6, consider training a perceptron with two features ($x_1, x_2$) and a bias term on the training data below. The training data is plotted for your convenience.

| $x_1$ | $x_2$ | **Label** |
|---|---|---|
| 3 | 4 | + |
| 1 | 3 | − |
| 2 | 3 | + |
| 4 | 1 | − |
| 1 | 2 | + |
| 2 | 4 | − |
| 3 | 1 | − |




,

Q6.4 (2 points) We initialize the perceptron with weight vector $w = [-1, 1, 0]$, where the weights correspond to features $x_1$, $x_2$, and 1 (bias feature), respectively.

After running one iteration of the perceptron algorithm on the first 3 data points in the table, what is the new weight vector?

Ⓐ $[-1, 1, 0]$ Ⓑ $[0, 4, 1]$ Ⓒ $[0, 1, 0]$ Ⓓ $[-4, -5, -2]$

**Solution:**

First point: $[3, 4, 1] \cdot [-1, 1, 0] = 1 \rightarrow$ correct

Second point: $[1, 3, 1] \cdot [-1, 1, 0] = 2 \rightarrow$ incorrect so $w = [-1, 1, 0] - [1, 3, 1] = [-2, -2, -1]$

Third point: $[2, 3, 1] \cdot [-2, -2, -1] = -11 \rightarrow$ incorrect so $w = [-2, -2, -1] + [2, 3, 1] = [0, 1, 0]$

Q6.5 (1 point) True or false: There exists some initial weight vector for which the perceptron algorithm is guaranteed to converge on the data points above.

Ⓐ True Ⓑ False

**Solution:** The data is not linearly separable, so there is no weight vector that correctly classifies all points. Since the perceptron algorithm converges/terminates once all data points are classified correctly, we will never converge no matter the initial weight vector.

Q6.6 (2 points) Which of the following features, if added **by itself**, would allow us to perfectly classify the data points above with a perceptron? There may be multiple answers; select all answers that work.

[A] $|x_1 - x_2|$ [B] $x_1^2$ [C] $x_1 + x_2$ Ⓓ None of the above

**Solution:** The positive points all have $|x_1 - x_2| = 1$ and negative points all have $|x_1 - x_2| > 1$, so accounting for the fact that we are using a bias term, that works.

Q6.7 (1 point) True or false: In general, we prefer using Naive Bayes over the perceptron algorithm for classification when our features take on continuous values. Assume that our data points are linearly separable by their labels.

Ⓐ True Ⓑ False

**Solution:** If features take on continuous values, representing the CPTs in Naive Bayes would be intractable since there would be infinite possible values for the features. Perceptrons can handle continuous feature values, on the other hand.

For Q6.8 and Q6.9, we switch from using a perceptron to using a decision tree to classify the same data.

Q6.8 (1 point) Consider a decision tree with one split:
All data points with $x_1 > 2$ and $x_2 < 2$ are classified as $+$. All other data points are classified as $-$.

What is the entropy of this split, given the training data above?

Ⓐ 0    ● B $-0.4\log(0.4)$    Ⓒ $-0.6\log(0.6)$    Ⓓ 1

**Solution:**

Update: None of the above are correct. This was fixed on the exam grades. The correct answer should be:

Entropy of the left branch is $-1\log(1) - 0\log(0) = 0$

Entropy of the right branch is $-\frac{2}{5}\log(\frac{2}{5}) - \frac{3}{5}\log(\frac{3}{5})$

Weighting by the number of data points in each branch gives us $\frac{2}{7} * 0 + \frac{5}{7} * [-\frac{2}{5}\log(\frac{2}{5}) - \frac{3}{5}\log(\frac{3}{5})]$

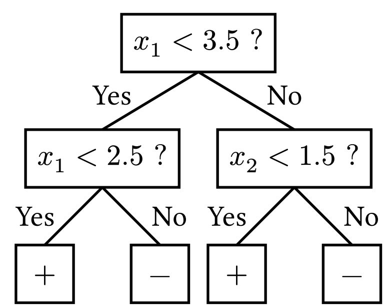
——————-

Old solution: Entropy is $\sum -p_i \log p_i$. The probability of $+y$ in the $x_1 > 2, x_2 < 2$ section is 1, and in the remaining section it is 2/5. $-1\log(1) - \frac{2}{5}\log(\frac{2}{5}) = -0.4\log(0.4)$.

Q6.9 (2 points) For this subpart, each decision boundary must split on whether a single feature is less than or greater than a specific value.

What is the minimum depth of a decision tree that correctly classifies all of the data points?

An example (incorrect) tree with depth 2 is shown.




Ⓐ 2    ● B 4    Ⓒ 6    Ⓓ 7

**Solution:** You can split up the points with 4 lines with each line being horizontal or vertical. For example, put one at $x_1 = 2.5$ and another at $x_1 = 1.5$. Put one line at $x_2 = 2.5$ and another at $x_2 = 3.5$. You will have perfectly split the data points.

Q6.10 (2 points) In general (not necessarily using the data above), consider building two trees using the same training data set: **Tree A** is limited to a finite depth, and **Tree B** has no depth limit.

Select all true statements about the two trees.

- [A] Tree A could have a strictly greater training accuracy than Tree B.
- [B] Tree A could have a strictly greater validation accuracy than Tree B.
- [C] Tree A could have a strictly greater test accuracy than Tree B.
- [D] Both trees could have the same number of leaves.
- (E) None of the above

**Solution:** Making the tree deeper can never decrease training accuracy.

Validation and test accuracy though have no guarantees. If Tree B overfits, then the validation and test accuracy for Tree B could be lower than Tree A.

If all data points in each leaf of Tree A have identical feature values, then increasing the depth would not result in any new leaf nodes.

Q6.11 (1 point) True or false: In general, decision trees can learn nonlinear decision boundaries.

(A) True   (B) False

**Solution:** Decision trees can learn piecewise-linear decision boundaries since each node can can split the feature space with a line.

## Q7 *Machine Learning: Deep RL* **(17 points)**

Consider an MDP with unknown transition probabilities and rewards.

In this question, we will compare three methods for learning Q-values in this MDP:
1. Standard (tabular) Q-learning from lecture.
2. Approximate Q-learning from lecture.
3. Build a neural network that learns the $Q$-function $Q(s, a)$.

In Q7.1 and Q7.2, assume the MDP has $|S|$ states and $|A|$ actions.

Q7.1 (1 point) How many parameters do we need to store when using standard Q-learning?

● (A) $|S| \cdot |A|$ ○ (B) $|S|^{|A|}$ ○ (C) $|S| + |A|$ ○ (D) None of the above

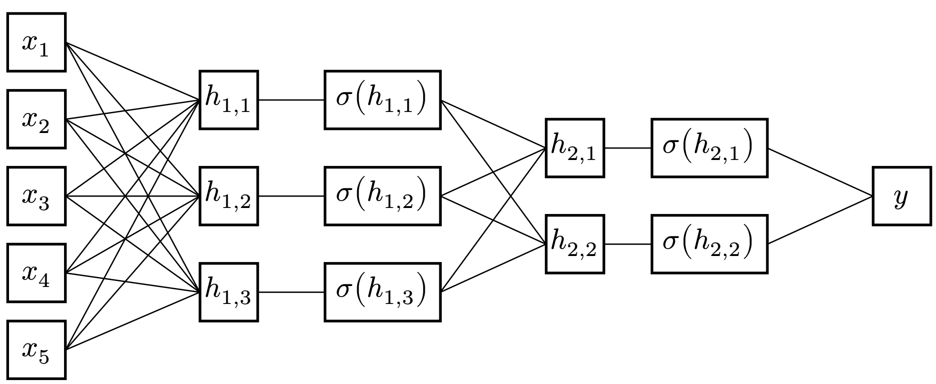
**Solution:** We need a table that stores one Q-value for every single $(s, a)$ pair. There are $|S|$ states and $|A|$ actions, so there are $|S| \cdot |A|$ total entries.

Q7.2 (1 point) How many parameters do we need to store when using approximate Q-learning? Given a state-action pair, we derive $n$ features from the state and $m$ features from the action.

○ (A) $|S| \cdot |A|$ ○ (B) $|S| \cdot |A| \cdot m \cdot n$ ● (C) $m + n$ ○ (D) $m \cdot n$

**Solution:** For approximate Q-learning, we store a parameter for every feature. Because there are $n$ features derived from states and $m$ features derived from actions, we know that there are $n + m$ parameters to store.

We design a neural network to approximate the $Q(s, a)$ function. The network has this architecture: 5-dimensional input, hidden layer with 3 neurons, hidden layer with 2 neurons, 1-dimensional output.




Q7.3 (1 point) How many total parameters do we need to store when using a neural network with these dimensions? (Ignore any bias terms.)

○ (A) 11 ○ (B) 15 ○ (C) 21 ● (D) 23 ○ (E) 28 ○ (F) None

**Solution:** Each edge represents a weight (parameter). Looking at the diagram, the first layer has 15 parameters, the second layer has 6 parameters, and the third layer has 2 parameters. Adding together, we get 23.

Q7.4 (1 point) How should we generate the inputs to the neural network?

- (A) [selected] Run a feature extraction function $f(s, a)$ to get a vector $[x_1, x_2, x_3, x_4, x_5]$.
- (B) Run a feature extraction function $f(s, a)$ to get a scalar $x_1 + x_2 + x_3 + x_4 + x_5$.
- (C) Run a feature extraction function $f(s, a)$ to get a scalar $y$.
- (D) Set each $x_i$ to a state-action pair $(s, a)$.
- (E) Set $y$ to the state-action pair $(s, a)$.

> **Solution:** In order to generate inputs to the neural network, we need to run a feature extraction function because neural networks require numeric vectors as inputs. We need utilize a feature extraction function to transform $(s, a)$ information into standardized numerical values. Scalar values are incorrect because they are too constrained and will lose information.

Q7.5 (1 point) Which value in the neural network represents the predicted Q-value?

- (A) [selected] $y$
- (B) $\sum_{i=1}^{5} x_i$
- (C) $y + \sum_{i=1}^{5} x_i$
- (D) $[x_1, x_2, x_3, x_4, x_5]$

> **Solution:** The question says that we are trying to approximate the Q-function, which is a function taking in $(s, a)$ as input and outputting $Q(s, a)$. In other words, the estimated Q-value is the output of the function.
>
> $y$ is the value outputted by the neural network.

Q7.6 (2 points) We are now training the neural network. $(s, a, s', r)$ represents a training data point, and $\hat{Q}(s, a)$ represents the neural network's predicted Q-values.

What happens when we train the neural network by minimizing the loss function below?

$$\text{Loss} = \left[\left(r + \gamma \max_{a'} \hat{Q}(s', a')\right) - \hat{Q}(s, a)\right]^2$$

- (A) The neural network will minimize Q-values for all $(s, a)$ pairs.
- (B) The neural network will maximize Q-values for all $(s, a)$ pairs.
- (C) The predicted Q-values will approximate the immediate reward of the state-action pair.
- (D) [selected] The predicted Q-values will satisfy the Bellman equation more closely.

> **Solution:** Minimizing the loss function promotes making predicted Q-values satisfy the Bellman equation more closely because $\left(r + \gamma \max_{a'} \hat{Q}(s', a')\right)$ represents the Bellman optimality condition and $\hat{Q}(s, a)$ is the network's current estimate. When we minimize their squared difference, then our network updates its parameters, and our predictions become more consistent with the Bellman equation.

Q7.7 (2 points) We train the neural network to convergence, and we notice that the neural network produces bad predicted Q-values.

Which modifications could help the network perform better? Consider each choice independently.

- [x] A Increase the number of hidden neurons in each layer.
- [x] B Change the feature extraction function.
- [ ] C Continue training the neural network on the existing training data.
- [x] D Provide additional training data that covers unseen $(s, a)$ pairs.
- (E) None of the above

**Solution:** Increasing the number of hidden features in each layer allows us to represent more complex relationships. Changing the feature extraction function could allow us to find better features that would make our inputs more structured. Providing additional training data allows for us to improve generalization and reach convergence on previously unseen $(s, a)$ pairs.

If we train on the existing data with the same learning rate, then if our neural network has already converged, we are unable to generalize further and could overfit our data.

Q7.8 (2 points) We train the neural network to convergence. Given a state $s$, how do we use the neural network to choose an optimal action?

- (A) Run the gradient descent algorithm.
- (B) Use the network to predict a single $\hat{Q}(s, a)$ value.
- (C) [selected] Use the network to predict multiple $\hat{Q}(s, a)$ values, then take an argmax.
- (D) Use the network to predict multiple $\hat{Q}(s, a)$ values, then take an average.

**Solution:** You would need to choose the action that gives you the highest predicted Q-value.

Q7.9 (3 points) Which algorithms can output predicted Q-values for unseen Q-states (i.e., Q-states that have never appeared in any sample)? Select all that apply.

- [x] A Neural networks
- [ ] B Standard tabular Q-learning
- [x] C Approximate Q-learning
- (D) None of the above

**Solution:** Neural nets utilize nonlinear functions to estimate Q-values for unseen states. Approximate Q-learning allows for us to estimate unseen states utilizing features to generalize.

Standard Q-learning uses an explicitly given table, so we cannot generalize to states that we have not seen yet.

Q7.10 (3 points) Which algorithms can learn Q-states in a continuous state space (i.e., an MDP with infinitely many states)? Select all that apply.

- [x] A Neural networks
- [ ] B Standard tabular Q-learning
- [x] C Approximate Q-learning
- D None of the above

**Solution:** Neural networks and approximate Q-learning can learn in continuous state space because they utilize function approximation and not a discrete look-up table. Because of this, they can both calculate any state through feature generalization.

In contrast, standard tabular Q-learning would require infinitely many entries, so our table would need to be infinitely large.

# Q8 *Project Potpourri* (6 points)

Q8.1 and Q8.2 are from Project 1 (Search).

In Project 1, you implemented the `ClosestDotSearchAgent`.

To solve the eat-all-dots problem, the `findPathToClosestDot` method in the `ClosestDotSearchAgent` class is called $\underset{\text{(i)}}{\_\_\_}$ time(s), and the resulting solution is $\underset{\text{(ii)}}{\_\_\_}$ optimal.

Q8.1 (1 point) Blank (i):

- (A) Exactly 1
- (B) Exactly 4
- (C) 1 or more

Q8.2 (1 point) Blank (ii):

- (A) Always
- (B) Sometimes
- (C) Never

**Solution:** For Q8.1 and Q8.2, see Question 8 from Project 1.

The agent calls the method once to find the closest dot, eats the dot, and then repeats for all the dots.

This might be the best path (ex: Pac-Man has one dot to eat), but it might also not be the optimal path.

Q8.3 (2 points) In Project 2 (Multi-Agent Search), when do we apply an evaluation function? Select all that apply.

- [A] After we return the value at the root.
- [B] When the current game state is a victory for Pac-Man.
- [C] When we have reached the maximum depth of the game tree.
- (D) None of the above. We never use an evaluation function in Project 2.

**Solution:** See Question 2 from Project 2.

We call **`self.evaluationFunction`** when **`gameState.isWin()`** or **`gameState.isLose()`** or **`depth == maxDepth`**.

Q8.4 (1 point) In Project 5 (Machine Learning), you implemented the `DigitClassificationModel` to classify handwritten digits from the MNIST dataset. What loss function do we use?

- (A) ReLU
- (B) Softmax
- (C) Mean Squared Error
- (D) Mean Absolute Error
- (E) Cross Entropy
- (F) Sigmoid

**Solution:** See Question 3 from Project 5.

ReLU, softmax, and sigmoid are not loss functions.

Mean Squared Error and Mean Absolute Error are loss functions for regression problems.

Cross entropy is the only option that is a loss function for a classification problem.

Q8.5 (1 point) In Project 5, you applied a causal mask to your attention layer. What was the primary purpose of using a causal mask in Project 5?

- (A) To prevent the neural network from looking ahead at future tokens in the sequence.
- (B) To prevent the neural network from looking back at past tokens in the sequence.
- (C) To ensure that our neural network only looks at the current token.
- (D) To ensure that our neural network does not overfit on the training data.

**Solution:** See Question 6 from Project 5.

Option (B) is incorrect because a transformer needs to look at past tokens in order to make a decision about the next token.

Option (C) is incorrect because we want our neural network to look at other tokens.

Option (D) is incorrect because overfitting has nothing to do with the future tokens in the sequence.